

BÁO CÁO NGHIÊN CỨU: ĐÁNH GIÁ KHẢ NĂNG SINH MÃ MỨC ĐỘ LỚP CỦA LLM TRÊN BENCHMARK CLASSEVAL

Người thực hiện: Nguyễn Cảnh Hào
GitHub: github.com/canhhao276/projectISL_Lab

Mô hình: Gemini 3.1 Flash Lite

1. GIỚI THIỆU (INTRODUCTION)

Báo cáo này trình bày kết quả đánh giá mô hình ngôn ngữ lớn (LLM) - Gemini 3.1 Flash Lite trên benchmark **ClassEval**. Khác với các bộ đánh giá mức hàm (function-level) truyền thống như HumanEval, ClassEval tập trung vào khả năng lập trình hướng đối tượng (OOP), yêu cầu mô hình phải hiểu mối quan hệ phức tạp giữa các phương thức và trạng thái nội tại của một lớp.

2. PHƯƠNG PHÁP NGHIÊN CỨU (METHODOLOGY)

2.1. Quy trình đánh giá (Pipeline)

Em xây dựng hệ thống tự động hóa hoàn chỉnh:

- Dữ liệu:** Lựa chọn 10 bài toán mẫu đầu tiên từ tập dữ liệu ClassEval
- Chiến lược sinh mã (RQ2 Focus):** Tập trung vào **Holistic Generation** (Sinh mã tổng thể). Đây là chiến lược phổ biến nhất trong thực tế khi các lập trình viên sử dụng AI: yêu cầu AI hoàn thiện toàn bộ một lớp từ khung xương (Skeleton) hoặc mô tả. Đây cũng là kịch bản thách thức nhất đối với khả năng duy trì logic nhất quán của mô hình.
- Prompting:** Thử nghiệm hai dạng: **Baseline** (Chỉ dẫn đơn giản) và **Structured** (Chỉ dẫn có cấu trúc).
- Chấm điểm:** Sử dụng `pytest` để chạy các bộ kiểm thử (test cases) trên mã nguồn AI đã viết, từ đó đo lường tỷ lệ thành công (Pass@1).
- Phân tích phụ thuộc (Dependency Analysis):** Thực hiện rà soát cấu trúc mã nguồn tự động để định lượng độ phức tạp của bài toán thông qua việc đếm số lượng lời gọi hàm nội bộ (Method Dep) và truy cập thuộc tính (Field Dep) từ mã nguồn chuẩn.

2.2. Các chỉ số đo lường

- Pass@1:** Tỷ lệ vượt qua toàn bộ Class-level tests trong một lần sinh.
- Method Requirements:** Số lượng lời gọi hàm nội bộ có trong mã nguồn chuẩn (đo độ phức tạp về hàm).
- Field Requirements:** Số lượng truy cập biến/thuộc tính có trong mã nguồn chuẩn (đo độ phức tạp về dữ liệu).

3. KẾT QUẢ THỰC NGHIỆM (RESULTS)

Danh sách chi tiết (Pass@1):

- PASSED:** Task 002, 004, 006, 007, 008, 009, 010.

Chỉ số đánh giá	Baseline Prompt	Structured Prompt
Tổng số Task	10	10
Số lượng PASSED (Class-level)	7	7
Chỉ số Pass@1	70.0%	70.0%

- **FAILED:** Task 001 (AccessGatewayFilter), 003 (ArgumentParser), 005 (AssessmentSystem).

4. PHÂN TÍCH LỖI CHI TIẾT (ERROR ANALYSIS - RQ4)

Trong quá trình thực nghiệm, mô hình hầu như không gặp lỗi cú pháp hay lỗi thiếu class, thiếu method. Phần lớn các lỗi thuộc các nhóm như **Logic Error**, **Wrong Return Value** và **TypeError** — đây cũng là các loại lỗi được đề cập trong bài **Evaluating Large Language Models in Class-Level Code Generation**.

Ở **task_001** (AccessGatewayFilter), mô hình vượt qua 17/19 test cases và thất bại ở 2 test cases (**test_filter_7**, **test_filter_8**). Các lỗi thuộc nhóm **Wrong Return Value** khi chương trình trả về kết quả khác với yêu cầu của bộ test. Điều này cho thấy mô hình hiểu được cấu trúc chương trình nhưng xử lý chưa đúng logic điều kiện trong một số trường hợp.

Ở **task_003** (ArgumentParser), mô hình vượt qua 17/21 test cases và thất bại ở 4 test cases (**test_parse_arguments_1**, **test_parse_arguments_2**, **test_convert_type_5**, **test_main**). Các lỗi thuộc nhóm **Logic Error** do chương trình xử lý sai chuỗi command đầu vào và cập nhật chưa đúng dữ liệu bên trong class. Ngoài ra, mô hình cũng xử lý chưa chính xác việc chuyển đổi kiểu dữ liệu boolean trong Python.

Ở **task_005** (AssessmentSystem), mô hình vượt qua 30/31 test cases và chỉ thất bại ở 1 test case (**test_get_course_average_2**). Lỗi thuộc nhóm **TypeError** kết hợp với **Logic Error** khi chương trình thực hiện phép tính với dữ liệu None mà không kiểm tra trước. Đây là lỗi thường gặp khi xử lý các trường hợp dữ liệu đặc biệt (edge case).

5. THẢO LUẬN (DISCUSSION)

5.1. Khả năng sinh mã mức Class (RQ1)

Mô hình Gemini 3.1 Flash Lite đạt hiệu năng khá tốt (**70% Pass@1 ở mức Class-level**). Tuy nhiên, khi nhìn sâu hơn vào kết quả ở mức độ phương thức (**Method-level/Test-case level**), mô hình cho thấy sự xuất sắc vượt trội:

- Ngay cả ở các Task bị tính là FAILED, AI thường vượt qua hầu hết các bài kiểm tra. Ví dụ: **Task 005 (vượt qua 30/31 test cases)** và **Task 003 (vượt qua 17/21 test cases)**.
- **Nhận xét:** Điều này chứng tỏ Gemini 3.1 có khả năng viết mã đơn lẻ cực kỳ chính xác. Sự thất bại ở mức Class-level (Pass@1) không phải do AI không biết viết code, mà do "lỗi tích tụ" hoặc sai sót ở một phương thức then chốt làm ảnh hưởng đến toàn bộ tính nhất quán của Class.

5.2. Ảnh hưởng của Prompting (RQ2)

Nghiên cứu tập trung vào chiến lược Holistic Generation (Sinh mã tổng thể), trong đó mô hình được yêu cầu sinh toàn bộ class trong một lần generate duy nhất. Đây là chiến lược phổ biến đối với các mô hình LLM có khả năng instruction-following tốt.

Trong quá trình thực nghiệm, việc thay đổi từ Baseline Prompt sang Structured Prompt trong cùng chiến lược Holistic không tạo ra sự khác biệt đáng kể về kết quả Pass@1 trên các task đã thử nghiệm.

Kết quả này cho thấy mô hình Gemini có xu hướng tập trung chủ yếu vào thông tin chức năng và skeleton của bài toán hơn là sự khác biệt nhỏ trong cách tổ chức prompt. Đồng thời, chiến lược Holistic vẫn cho thấy khả năng sinh mã tương đối ổn định đối với các bài toán class-level trong phạm vi thực nghiệm của project.

Tuy nhiên, do số lượng task và quy mô thực nghiệm còn hạn chế, kết quả hiện tại chưa đủ để khẳng định tính vượt trội của Holistic Generation so với các chiến lược khác như Incremental Generation.

5.3. Phân tích phụ thuộc (RQ3 - Dependency Analysis)

Dựa trên việc phân tích mã nguồn chuẩn (`solution.py`), Em xem xét mức độ phụ thuộc giữa các thành phần trong class, bao gồm: **phụ thuộc giữa các method** (Method Requirements) và **phụ thuộc giữa các fields/attributes** (Field Requirements).

Kết quả thực nghiệm cho thấy một số điểm đáng chú ý sau:

- **Độ phức tạp của Method Dependency:** Các bài toán có mức phụ thuộc method cao hơn, như `Task_001` và `Task_003`, thường xuất hiện nhiều lỗi hơn. Các lỗi chủ yếu liên quan đến việc gọi hàm, cập nhật trạng thái dữ liệu và duy trì logic nhất quán giữa nhiều method trong cùng một class.
- **Sự ổn định của Field Dependency:** Ngược lại, các bài toán có mức phụ thuộc thấp hoặc chủ yếu tập trung vào thao tác trên fields/attributes như `Task_002` và `Task_009` cho kết quả ổn định hơn và đạt tỷ lệ pass cao hơn.
- **Yếu tố Edge Case:** Ngoài ra, ở `Task_005`, mặc dù số lượng method dependency không lớn, mô hình vẫn thất bại ở một trường hợp edge case liên quan đến dữ liệu `None`. Điều này cho thấy ngoài *dependency complexity*, khả năng xử lý các trường hợp đặc biệt cũng ảnh hưởng đáng kể đến chất lượng mã nguồn được tạo ra.

Kết luận: Từ các kết quả trên, có thể thấy mô hình gặp nhiều khó khăn hơn khi phải phối hợp logic giữa nhiều method trong class. Quan sát này tương đối phù hợp với **Finding 4** của bài báo gốc, trong đó *method-level dependency* được xem là một trong những thách thức lớn nhất đối với *class-level code generation*.

5.4. Phân tích lỗi chi tiết (RQ4 - Bad Case Analysis)

Em thực hiện phân tích các trường hợp thất bại để hiểu rõ hơn giới hạn của mô hình trong *class-level code generation*. Các lỗi xuất hiện chủ yếu thuộc nhóm **Logic Error**, **Wrong Return Value** và **TypeError**, tương tự các nhóm lỗi được đề cập trong bài báo gốc.

Trường hợp tiêu biểu là `Task_005`, nơi mô hình gặp lỗi:

```
TypeError: unsupported operand type(s) for +=: 'int' and 'NoneType'
```

Lỗi xảy ra khi chương trình thực hiện phép cộng với một giá trị `None` trong quá trình tính điểm trung bình môn học. Nguyên nhân là mô hình không kiểm tra dữ liệu đầu vào trước khi thực hiện phép tính.

Ngoài ra, ở `Task_001` và `Task_003`, mô hình cũng xuất hiện các lỗi liên quan đến xử lý logic điều kiện, cập nhật trạng thái dữ liệu và duy trì tính nhất quán giữa nhiều method trong cùng một class.

Các kết quả này tương đối phù hợp với **Finding 5** của bài báo gốc, cho thấy LLM vẫn còn hạn chế trong việc xử lý *semantic constraints* và *state dependencies* trong các bài toán class-level phức tạp.

6. KẾT LUẬN & HƯỚNG PHÁT TRIỂN

Dự án đã xây dựng thành công một pipeline đánh giá cho bài toán class-level code generation dựa trên benchmark ClassEval. Kết quả thực nghiệm cho thấy mô hình Gemini 3.1 Flash Lite có thể sinh ra các method và xử lý tốt nhiều chức năng riêng lẻ ở mức function-level, thể hiện qua tỉ lệ pass cao ở phần lớn các test cases.

Tuy nhiên, khi bài toán yêu cầu nhiều method phải phối hợp với nhau trong cùng một class, mô hình bắt đầu xuất hiện các lỗi về logic, cập nhật trạng thái dữ liệu và xử lý các trường hợp đặc biệt. Điều này cho thấy mặc dù LLM hiện nay đã khá hiệu quả ở function-level code generation, khả năng reasoning ở class-level vẫn còn hạn chế, đặc biệt trong việc duy trì sự nhất quán giữa các method và field bên trong class.

Kết quả thực nghiệm của project tương đối phù hợp với kết luận của bài báo Evaluating Large Language Models in Class-Level Code Generation rằng class-level code generation là một thách thức khó hơn đáng kể so với function-level generation đối với các mô hình ngôn ngữ lớn.

Hướng phát triển: Mở rộng sang các chiến lược **Incremental** và **Compositional Generation** để đánh giá sự thay đổi hiệu năng khi chia nhỏ bài toán.

Chữ ký xác nhận

Nguyễn Cảnh Hào